FOP MINI PROJECT

TEAM MEMBERS:-

SAKSHAM BISEN	1262253617
PARTH SINGHAL	1262253370
PARV JAIN	1262253196

PROBLEM STATEMENT

SOLVING THE ISSUE OF CAMPUS CAFE INEFFICIENCY

The Encave cafes in our campus are the one stop solution for having a quick meal or hanging out with your friends and colleagues. It is accessed by many students and faculty members all through out the day. It’s location on the campus really makes it the best spot for having lunch in-between lectures.

All this sounds good on the surface but the experience in the cafes is not what we would expect it to be. Long waiting time just to order food and even longer waiting time for getting our order, and sometimes even if we get the chance to order food that we want, we can’t order it because of unavailability of raw materials to make that order. All this makes going to Encave cumbersome and full of hassle.

THE SOLUTION

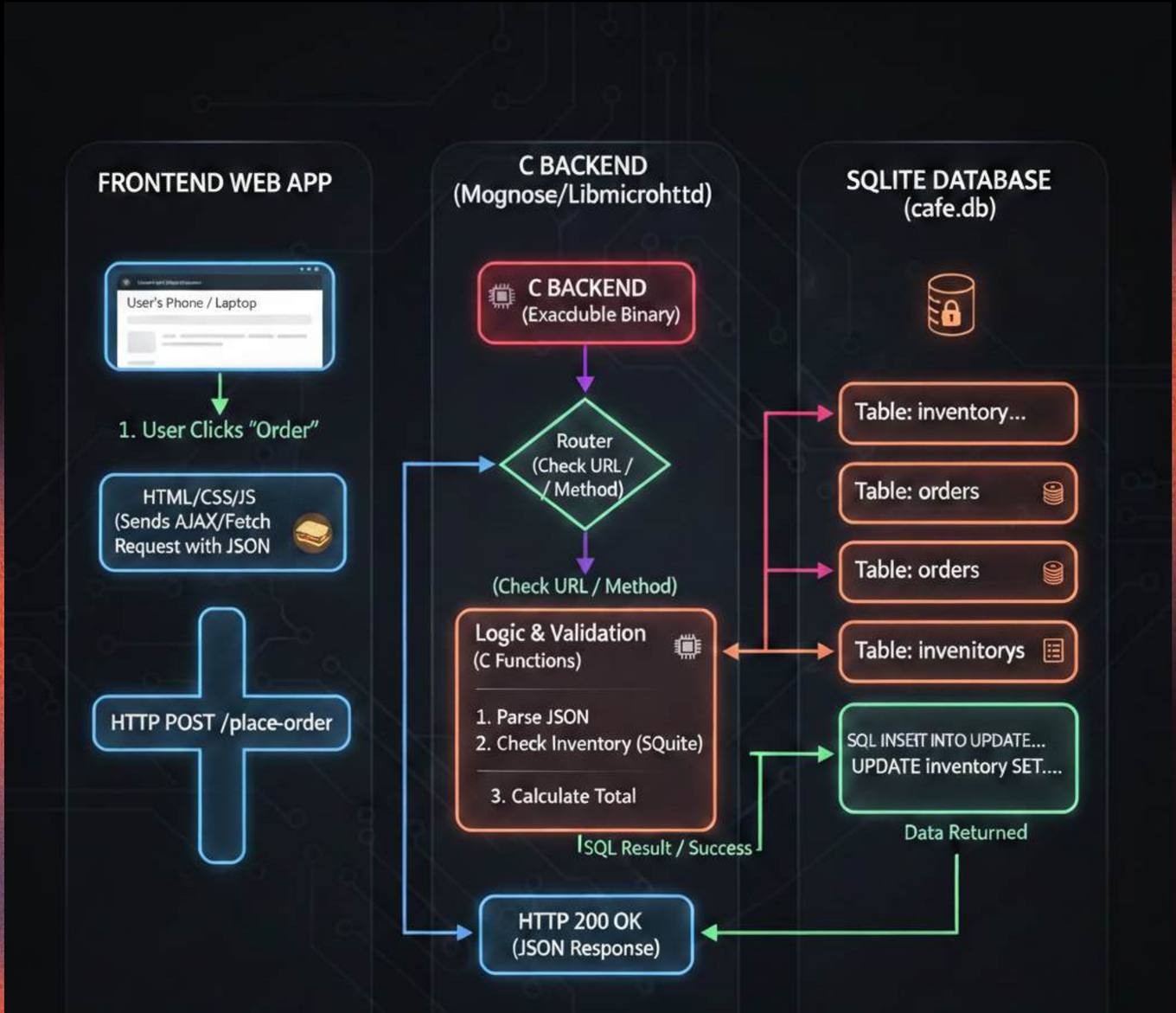
We wanted to use our knowledge of Computer Science to create something innovative that could help us solve the problem with the cafe as it is very relevant to our daily life.

Our idea is to create A High-Performance Resource Management & Ordering System.

Firstly creating a database for managing all the resources and raw materials so that it will be easy to know what items form the menu can be made.

Next up is the web app for ordering food, this will stop the place from getting overcrowded and make it much easier to take our orders.

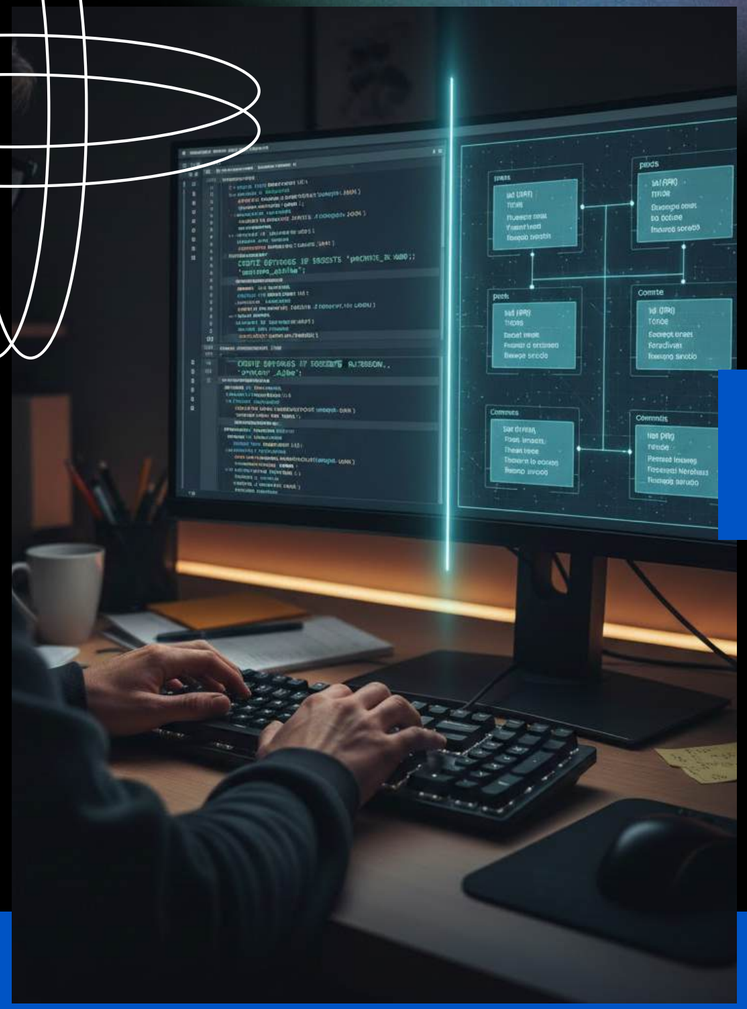
FLOWCHART


FRONTEND WEB APP
User's Phone / Laptop
1. User Clicks "Order"
HTML/CSS/JS
(Sends AJAX/Fetch
Request with JSON
HTTP POST /place-order
C BACKEND
(Mognose/Libmicrohttd)
C BACKEND
(Exacduble Binary)
Router
(Check URL /
/ Method)
(Check URL / Method)
Logic & Validation
(C Functions)
1. Parse JSON
2. Check Inventory (SQuite)
3. Calculate Total
SQL Result / Success
HTTP 200 OK
(JSON Response)
SQLITE DATABASE
(cafe.db)
Table: inventory...
Table: orders
Table: orders
Table: invenitorys
SQL INSEIT INTO UPDATE...
UPDATE inventory SET....
Data Returned

PART-1 THE BACKEND

For the backend we plan to use C programming language paired with the Mongoose library. What this does is after a person places an order, a signal will be received by our C code which will separate the order and name of the person from the signal and add these details to the database.

The reason we prefer C over Python or Node.js is because of its high efficiency and ability to run on Low-power hardware like a raspberry pi.



```sql
CREATE TABLE users (
   user_id INT PRIMARY KEY AUTO_INCREMENT,
   username VARCHAR(50) NOT NULL,
   email    VARCHAR(100) NOT NULL,
   created_at TIMESTAMP DEFAULT  CURRENT_TIMESTAMP;
);
CREATE TABLE products (
   product_id INT PRIMARY KEENUT_INCREMENT,
   product_name VARCHAR(100) NOT NULL,
   price DECIMAL(10,2) NOT NULL,
   creud  INT TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
);
CREATE TABLE orders (
   order_id INT PRIMARY KEY AUTO_INCREMENT,
   user_id INT NOT NULL,
   order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
);
CREATE TABLE order_items (
   order_item_id INT PRIMARY KEY AUTO_INCREMENT;
   order_id INT NOT NULL,
   product_id INT NOT NULL,
   quantity INT NOT NULL,
   subtotal DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (order_id) REFERENCES orders(order_id),
   FOREIGN KEY (product_id) REFERENCES products(prdcr_luid);
);
```

PART-2 THE DATABASE

The database will have three tables in total. First will be the inventory which will reflect if a particular dish can be ordered on the website or not depending upon the resources.

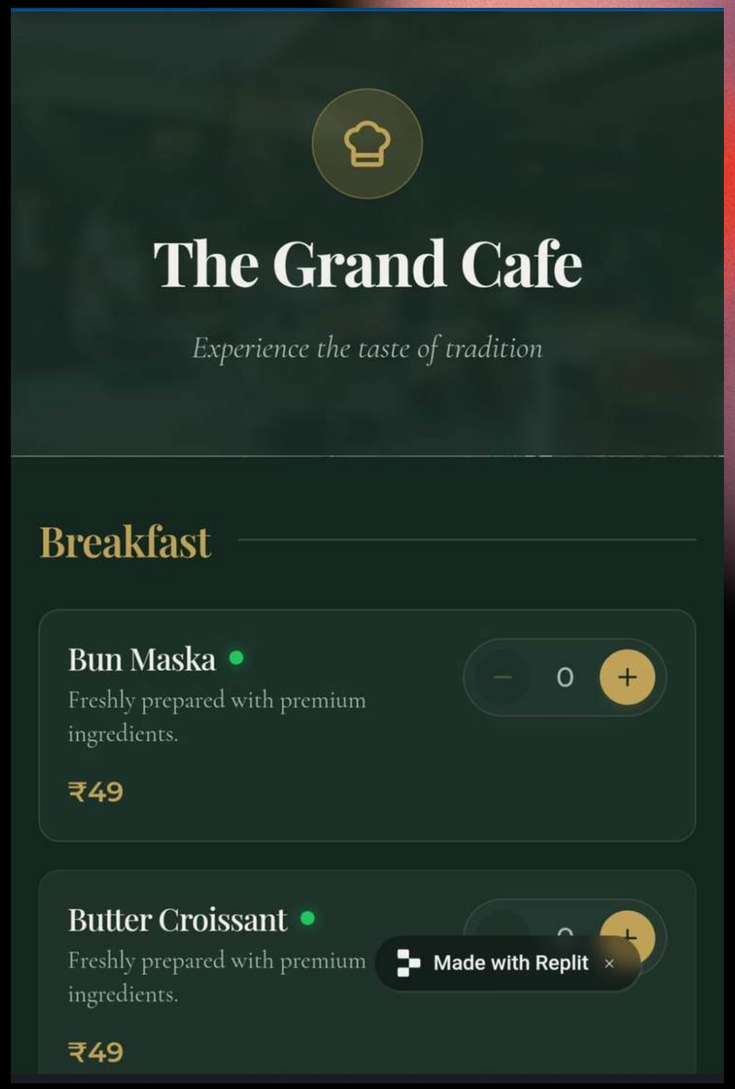
Next is the orders table which will contain details of the order including Name, PRN or Unique College ID Code and items ordered.

Lastly there will be a items table which will contain the items from the menu and their price. This table will link the items to the main order.

PART-3 THE WEB APP

To make the ordering process more simple we have created a web app that can be accessed using the campus Wi-Fi.

For the front-end we have used HTML, CSS and JavaScript.
HTML for basic layout of the web app
CSS for design of the web app
JavaScript for connecting our front-end and database.



TERMS AND CONDITIONS

1

To prevent students from exploiting this system we can charge them half of the billing amount at the time of placing the order.

We can implement a Status feature which will not accept the order until payment status is verified.

Once the order is placed it cannot be cancelled.

2

There will be a strike system implemented which will flag any user who has missed food pick-ups.

After a certain number of flags the user would be temporarily blocked from the system.

3

A unique QR Code generated at the time of booking must be shown at the time of food pickup.

A valid College ID or Guest Session ID (For people who don't have an ID) is mandatory for food pick-up.

FUTURE IMPLEMENTATIONS

In the future we plan on expanding this project by integrating it with PREDICTIVE ANALYTICAL Models to analyze our orders and suggest which items are the most frequently ordered.

We can also start a delivery system that delivers our food to the required building so students don't have to walk all the way to the cafe. For this we can use the infrastructure that we already have like golf carts that makes round on our campus.

THANK YOU!